CTCLoss#

https://pytorch.org/docs/stable/generated/torch.nn.CTCLoss.html

Description:

The Connectionist Temporal Classification loss. Calculates loss between a continuous (unsegmented) time series and a target sequence. CTCLoss sums over the probability of possible alignments of input to target, producing a loss value which is differentiable with respect to each input node. The alignment of input to target is assumed to be “many-to-one”, which limits the length of the target sequence such that it must be ≤\leq≤ the input length. blank (int, optional) – blank label. Default 000. reduction (str, optional) – Specifies the reduction to apply to the output: 'none' | 'mean' | 'sum'. 'none': no reduction will be applied, 'mean': the output losses will be divided by the target lengths and then the mean over the batch is taken, 'sum': the output losses will be summed. Default: 'mean' zero_infinity (bool, optional) – Whether to zero infinite losses and the associated gradients. Default: False Infinite losses mainly occur when the inputs are too short to be aligned to the targets.

Function Signature

```
class torch.nn.CTCLoss(blank=0, reduction='mean',
zero_infinity=False)[source]#

Parameters

Shape:

Reference:

forward(log_probs, targets, input_lengths, target_lengths)
[source]#
```

Returns:

Tensor

Code Examples

```
>>> # Target are to be padded
>>> T = 50  # Input sequence length
>>> C = 20  # Number of classes (including blank)
>>> N = 16  # Batch size
>>> S = 30  # Target sequence length of longest target in batch
(padding length)
>>> S_min = 10  # Minimum target length, for demonstration
purposes
>>>
>>> # Initialize random batch of input vectors, for *size =
```

```
(T,N,C)
>>> input = torch.randn(T, N,
C).log_softmax(2).detach().requires_grad_()
>>>
>>> # Initialize random batch of targets (0 = blank, 1:C =
classes)
>>> target = torch.randint(low=1, high=C, size=(N, S),
dtype=torch.long)
>>>
>>> input_lengths = torch.full(size=(N,), fill_value=T,
dtype=torch.long)
>>> target_lengths = torch.randint(
...     low=S_min,
...     high=S,
...     size=(N,),
...     dtype=torch.long,
... )
>>> ctc_loss = nn.CTCLoss()
>>> loss = ctc_loss(input, target, input_lengths,
target_lengths)
>>> loss.backward()
>>>
>>>
>>> # Target are to be un-padded
>>> T = 50  # Input sequence length
>>> C = 20  # Number of classes (including blank)
>>> N = 16  # Batch size
>>>
>>> # Initialize random batch of input vectors, for *size =
(T,N,C)
>>> input = torch.randn(T, N,
C).log_softmax(2).detach().requires_grad_()
>>> input_lengths = torch.full(size=(N,), fill_value=T,
dtype=torch.long)
>>>
>>> # Initialize random batch of targets (0 = blank, 1:C =
classes)
>>> target_lengths = torch.randint(low=1, high=T, size=(N,),
```

```
dtype=torch.long)
>>> target = torch.randint(
...     low=1,
...     high=C,
...     size=(sum(target_lengths),),
...     dtype=torch.long,
... )
>>> ctc_loss = nn.CTCLoss()
>>> loss = ctc_loss(input, target, input_lengths,
target_lengths)
>>> loss.backward()
>>>
>>>
>>> # Target are to be un-padded and unbatched (effectively N=1)
>>> T = 50  # Input sequence length
>>> C = 20  # Number of classes (including blank)
>>>
>>> # Initialize random batch of input vectors, for *size =
(T,C)
>>> input = torch.randn(T,
C).log_softmax(1).detach().requires_grad_()
>>> input_lengths = torch.tensor(T, dtype=torch.long)
>>>
>>> # Initialize random batch of targets (0 = blank, 1:C =
classes)
>>> target_lengths = torch.randint(low=1, high=T, size=(),
dtype=torch.long)
>>> target = torch.randint(
...     low=1,
...     high=C,
...     size=(target_lengths,),
...     dtype=torch.long,
... )
>>> ctc_loss = nn.CTCLoss()
>>> loss = ctc_loss(input, target, input_lengths,
target_lengths)
>>> loss.backward()
```

Notes & Warnings

NOTE Note In order to use CuDNN, the following must be satisfied: targets must be in concatenated format, all input_lengths must be T. blank=0blank=0blank=0, target_lengths ≤256\leq 256≤256, the integer arguments must be of dtype torch.int32. The regular implementation uses the (more common in PyTorch) torch.long dtype.

NOTE Note In some circumstances when using the CUDA backend with CuDNN, this operator may select a nondeterministic algorithm to increase performance. If this is undesirable, you can try to make the operation deterministic (potentially at a performance cost) by setting torch.backends.cudnn.deterministic = True. Please see the notes on Reproducibility for background.

Detailed Documentation

Documentation

The Connectionist Temporal Classification loss.

Calculates loss between a continuous (unsegmented) time series and a target sequence. CTCLoss sums over the probability of possible alignments of input to target, producing a loss value which is differentiable with respect to each input node. The alignment of input to target is assumed to be “many-to-one”, which limits the length of

the target sequence such that it must be ≤\leq≤ the input length.

blank (int, optional) – blank label. Default 000.

reduction (str, optional) – Specifies the reduction to apply to the output: 'none' | 'mean' | 'sum'. 'none': no reduction will be applied, 'mean': the output losses will be divided by the target lengths and then the mean over the batch is taken, 'sum': the output losses will be summed. Default: 'mean'

zero_infinity (bool, optional) – Whether to zero infinite losses and the associated gradients. Default: False Infinite losses mainly occur when the inputs are too short to be aligned to the targets.

Log_probs: Tensor of size (T,N,C)(T, N, C)(T,N,C) or (T,C)(T, C)(T,C), where T=input lengthT = \text{input length}T=input length, N=batch sizeN = \text{batch size}N=batch size, and C=number of classes (including blank)C = \text{number of classes (including blank)}C=number of classes (including blank). The logarithmized probabilities of the outputs (e.g. obtained with torch.nn.functional.log_softmax()).

Targets: Tensor of size (N,S)(N, S)(N,S) or (sum(target_lengths))(\operatorname{sum}(\text{target_lengths}))(sum(target_lengths)), where N=batch sizeN = \text{batch size}N=batch size and S=max target length, if shape is (N,S)S = \text{max target length, if shape is } (N, S)S=max target length, if shape is (N,S). It represents the target sequences. Each element in the target sequence is a class index. And the target index cannot be blank (default=0). In the (N,S)(N, S)(N,S) form, targets are padded to the length of the longest sequence, and stacked. In the (sum(target_lengths))(\operatorname{sum}(\text{target_lengths}))(sum(target_lengths)) form, the targets are assumed to be un-padded and concatenated within 1 dimension.

Input_lengths: Tuple or tensor of size (N)(N)(N) or ()()(), where N=batch sizeN = \text{batch size}N=batch size. It represents the lengths of the inputs (must each be

≤T\leq T≤T). And the lengths are specified for each sequence to achieve masking under the assumption that sequences are padded to equal lengths.

Target_lengths: Tuple or tensor of size (N)(N)(N) or ()()(), where N=batch sizeN = \text{batch size}N=batch size. It represents lengths of the targets. Lengths are specified for each sequence to achieve masking under the assumption that sequences are padded to equal lengths. If target shape is (N,S)(N,S)(N,S), target_lengths are effectively the stop index sns_nsn for each target sequence, such that target_n = targets[n,0:s_n] for each target in a batch. Lengths must each be ≤S\leq S≤S If the targets are given as a 1d tensor that is the concatenation of individual targets, the target_lengths must add up to the total length of the tensor.

Output: scalar if reduction is 'mean' (default) or 'sum'. If reduction is 'none', then (N)(N)(N) if input is batched or ()()() if input is unbatched, where N=batch sizeN = \text{batch size}N=batch size.

A. Graves et al.: Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks: https://www.cs.toronto.edu/~graves/icml_2006.pdf

In order to use CuDNN, the following must be satisfied: targets must be in concatenated format, all input_lengths must be T. blank=0blank=0blank=0, target_lengths ≤256\leq 256≤256, the integer arguments must be of dtype torch.int32.

The regular implementation uses the (more common in PyTorch) torch.long dtype.

In some circumstances when using the CUDA backend with CuDNN, this operator may select a nondeterministic algorithm to increase performance. If this is undesirable, you can try to make the operation deterministic (potentially at a performance cost) by setting torch.backends.cudnn.deterministic = True. Please see the notes on Reproducibility for background.

Runs the forward pass.